

مستند پروژه هوش مصنوعی

Intelligent Sports Match Scheduling (CSP)

توضیح عنوان

موضوع پروژه	پیاده‌سازی سیستم زمان‌بندی هوشمند مسابقات ورزشی با استفاده از مدل‌سازی مسئله ارضای محدودیت (CSP)
درس	هوش مصنوعی
زبان پیاده‌سازی	Python
اعضای تیم	دانشجو ۱ و دانشجو ۲

فهرست مطالب

۱. معرفی پروژه
۲. هدف و مسئله اصلی
۳. تقسیم کار دو دانشجو
۴. ساختار فایل‌های پروژه
۵. توضیح ورودی
۶. مدل‌سازی CSP: متغیر، دامنه و همسایگی
۷. الگوریتم Backtracking Search و Forward Checking
۸. هیوریستیک‌های امتیازی: LCV و MRV
۹. محدودیت مسابقات حساس و پیش‌بررسی امکان‌پذیری
۱۰. آزمایش: روش اجرا و نتایج
۱۱. نکات مهم برای ارائه
۱۲. جمع‌بندی
۱۳. پیوست: خلاصه توابع مهم

۱ معرفی پروژه

این پروژه با هدف طراحی و پیاده‌سازی یک سیستم زمان‌بندی هوشمند برای مسابقات ورزشی هفتگی انجام شده است. در چنین مسئله‌ای، باید به هر مسابقه یک سه‌تایی (روز، ساعت، استادیوم) اختصاص یابد؛ به‌گونه‌ای که هیچ تیمی در یک روز بیش از یک بازی نداشته باشد، هیچ استادیومی در یک زمان میزبان بیش از یک مسابقه نباشد، و مسابقات حساس (بخش امتیازی) در یک روز برگزار نشوند. این مسئله به‌صورت یک مسئله ارضای محدودیت (CSP) مدل‌سازی و با Backtracking Search همراه با Forward Checking حل شده است. به‌عنوان بخش امتیازی، هیوریستیک‌های LCV و MRV نیز اضافه شده‌اند.

۲ هدف و مسئله اصلی

هدف اصلی پروژه این است که نشان دهد چگونه یک مسئله واقعی زمان‌بندی را می‌توان با فرمول‌بندی متغیر/دامنه/محدودیت به CSP تبدیل کرد و با جست‌وجوی سیستماتیک (نه حدس تصادفی) حل کرد. ورودی پروژه یک فایل متنی است و برنامه بدون دریافت هیچ آرگومان اضافه‌ای، کل فرایند خواندن، مدل‌سازی و حل را انجام می‌دهد.

بخش	توضیح
ورودی	فایل متنی شامل تعداد استادیوم‌ها، روزها، ساعت‌ها، لیست مسابقات و مسابقات حساس
خروجی	انتساب (روز، ساعت، استادیوم) به هر مسابقه، یا No Solution؛ به‌همراه تعداد Backtracks و زمان اجرا
هدف محاسباتی	یافتن یک انتساب کامل که همه محدودیت‌ها را ارضا کند (نه لزوماً بهینه‌ترین)
زبان و مواد	Python، پیاده‌سازی دستی الگوریتم‌های جست‌وجو بدون استفاده از کتابخانه‌های آماده

۳ تقسیم کار دو دانشجو

برای اینکه پروژه منظم و قابل ارائه باشد، کار بین دو دانشجو تقسیم شده است. این تقسیم‌بندی باعث می‌شود هر دانشجو روی بخش مشخصی از مدل‌سازی، الگوریتم و توضیح آن مسلط باشد.

عضو تیم	فایل‌های مربوطه	مسئولیت اصلی
دانشجو ۱	input_reader.py csp_model.py	خواندن و پارس ورودی؛ مدل‌سازی CSP (متغیر، دامنه، همسایگی، تابع سازگاری) و پیش‌بررسی امکان‌پذیری
دانشجو ۲	main.py, solver.py	پیاده‌سازی Backtracking, Forward Checking, Search, MRV، LCV و نقطه ورود برنامه

۴ ساختار فایل‌های پروژه

فایل `main.py` نقطه شروع اجرای برنامه است و پروژه به چند فایل جداگانه تقسیم شده تا خوانایی و نگهداری کد بهتر شود.

نام فایل	کاربرد
<code>main.py</code>	دریافت مسیر فایل ورودی از خط فرمان و اجرای کامل مسیر حل مسئله
<code>input_reader.py</code>	خواندن فایل ورودی و تبدیل آن به ساختار داده (استادیوم‌ها، روزها، ساعت‌ها، مسابقات، حساس)
<code>csp_model.py</code>	ساخت متغیرها، دامنه‌ها، گراف همسایگی، تابع سازگاری و بررسی امکان‌پذیری
<code>solver.py</code>	پیاده‌سازی Forward Checking + Backtracking LCV/MRV +
<code>sample_data.py</code>	داده و توابع نمونه دستی، صرفاً برای تست اولیه <code>solver.py</code> در حین توسعه (جدا از خط اصلی اجرا)
<code>test_reader.py</code> <code>test_model.py</code> <code>test_solver.py</code>	اسکرپت‌های دستی برای تست مجزای هر ماژول
<code>tests/*.txt</code>	فایل‌های ورودی نمونه برای اجرای آزمایش‌ها

۵ توضیح ورودی

فایل ورودی به صورت خطی خوانده می‌شود؛ هر عدد یا نام، یک توکن مستقل است.

بخش ورودی	معنی
S + نام استادیوم‌ها	تعداد استادیوم‌ها و نام هرکدام
D	تعداد روزهای در دسترس
H	تعداد بازه‌های زمانی هر روز
$N + N$ خط	تعداد مسابقات و جفت‌تیم هر مسابقه (Team1 Team2)
$K + K$ خط	تعداد مسابقات حساس و جفت‌تیم هرکدام (ترتیب نام تیم‌ها اهمیتی ندارد)

۶ مدل‌سازی CSP: متغیر، دامنه و همسایگی

در سناریوی اصلی پروژه، مسئله به سه رکن متغیر، دامنه و محدودیت تجزیه می‌شود. هر مسابقه یک متغیر است؛ دامنه اولیه هر متغیر شامل همه سه‌تایی‌های (روز، ساعت، استادیوم) است؛ و چون دامنه اولیه همه مسابقات یکسان است، همه مسابقات با یکدیگر همسایه در نظر گرفته می‌شوند.

ویژگی	توضیح
متغیر	هر مسابقه (اندیس 0 تا $N-1$) - تابع build_variables
دامنه	همه سه تایی‌های (روز، ساعت، استادیوم)؛ اندازه = $S \times D \times H$ - تابع build_domains
همسایگی	گراف کامل (هر دو مسابقه همسایه‌اند) - تابع build_neighbors
تابع سازگاری	is_consistent: بررسی سه قانون (رست تیم، تداخل استادیوم، مسابقات حساس هم‌روز)
مزیت	سادگی پیاده‌سازی و درستی تضمین‌شده (چون دامنه اولیه برای همه یکسان است)
محدودیت	چند بررسی سازگاری اضافی و بی‌ضرر بین مسابقاتی که در عمل ربطی به هم ندارند

۷ الگوریتم Checking Forward و Search Backtracking

هسته حل‌کننده، جست‌وجوی عمقی روی درخت انتساب‌های جزئی است. پس از هر تخصیص، دامنه مسابقات هنوز مقداردهی‌نشده بلافاصله هرس می‌شود (Forward Checking)؛ اگر دامنه یکی از آن‌ها خالی شود، بلافاصله backtrack انجام می‌شود.

ویژگی	توضیح
الگوریتم	Forward Checking + Backtracking Search
حالت پایه	تعداد مسابقات مقداردهی‌شده = تعداد کل مسابقات (تابع backtrack_search)
شرط بازگشت	دامنه یک همسایه مقداردهی‌نشده پس از forward_check خالی شود
شمارنده Backtracks	هر بار که مقداری واقعاً تخصیص داده شده ولی به جواب کامل نرسیده، لغو می‌شود
مزیت	کشف زودهنگام بن‌بست؛ کاهش محسوس تعداد backtrack‌ها نسبت به Backtracking خام
محدودیت	در بدترین حالت نظری همچنان نمایی است (CSP در حالت کلی NP-Complete است)

مراحل کلی اجرا: انتخاب متغیر بعدی (بخش ۸) → مرتب‌سازی مقادیر دامنه (بخش ۸) → بررسی سازگاری با انتساب فعلی → تخصیص موقت → Forward Checking روی همسایگان → ادامه بازگشتی یا لغو تخصیص.

```
python main.py tests/sample_bonus.txt
```

```

Team_A Team_B 1 1 Azadi
Team_C Team_D 2 1 Azadi
Team_A Team_C 3 1 Azadi
Team_B Team_D 3 1 Takhti
Team_E Team_F 1 1 Takhti
Backtracks: 0
Time: 0.000506 seconds

```

۸ هیوریستیک‌های امتیازی: LCV و MRV

به‌عنوان بخش امتیازی، دو هیوریستیک استاندارد CSP به حل‌کننده اضافه شده‌اند تا تعداد backtrack‌ها کاهش یابد.

هیوریستیک	توضیح
Minimum) MRV Remaining (Values	انتخاب متغیر با کوچک‌ترین دامنه باقی‌مانده – تابع select_unassigned_variable
Least) LCV Constraining (Value	اولویت با مقداری که کمترین محدودیت را روی همسایگان ایجاد می‌کند – توابع order_values_lcv, count_removed_values
مزیت MRV	متغیرهای سخت‌تر زودتر بررسی می‌شوند؛ شکست احتمالی نزدیک ریشه درخت کشف می‌شود
مزیت LCV	دامنه متغیرهای آینده تا حد امکان باز می‌ماند؛ احتمال موفقیت بدون backtrack افزایش می‌یابد

این دو تکنیک در پس‌زمینه همان اجرای بخش قبل («دَب: Backtracks: ۰») فعال هستند؛ برای ورودی‌های بزرگ‌تر و سخت‌تر، تفاوت آن‌ها با حالت بدون هیوریستیک در تعداد Backtracks گزارش‌شده مشهودتر است.

۹ محدودیت مسابقات حساس و پیش‌بررسی امکان‌پذیری

علاوه بر دو محدودیت اصلی (رست تیم و تداخل استادیوم)، قانون سوم برای مسابقات حساس اضافه شده است. همچنین دو پیش‌بررسی ساده $O(1)$ پیش از شروع جست‌وجو انجام می‌شود تا در دو حالت غیرممکن، بلافاصله No Solution گزارش شود.

بررسی	توضیح
محدودیت مسابقات حساس	هیچ دو مسابقه حساس نباید در یک روز باشند – در تابع <code>is_consistent</code>
بررسی ظرفیت کل	اگر $N > S \times D \times H$ باشد، مسئله غیرممکن است
اصل لانه کبوتری	اگر $K > D$ باشد (مسابقات حساس بیشتر از روزها)، مسئله غیرممکن است
تابع مسئول	<code>is_feasible</code> – فراخوانی شده در <code>main.py</code> پیش از هرگونه جست و جو

```
python main.py tests/no_solution_capacity.txt
No Solution
Backtracks: 0
Time: 0.000000 seconds
```

```
python main.py tests/no_solution_sensitive.txt
No Solution
Backtracks: 0
Time: 0.000000 seconds
```

۱۰ آزمایش: روش اجرا و نتایج

برای اجرای پروژه از خط فرمان استفاده می‌شود. کاربر باید مسیر فایل ورودی را مشخص کند؛ قالب کلی دستور اجرا به شکل زیر است:

```
python main.py <input_file>
```

فایل ورودی	نتیجه اصلی
<code>sample_bonus.txt</code>	جواب معتبر پیدا شد؛ Backtracks: 0
<code>no_solution_capacity.txt</code>	No Solution فوری (بررسی ظرفیت کل)
<code>no_solution_sensitive.txt</code>	No Solution فوری (اصل لانه کبوتری)

با اجرای برنامه روی نمونه رسمی صورت پروژه (۲ استادیوم، ۲ روز، ۲ ساعت، ۴ مسابقه)، خروجی برنامه دقیقاً با یکی از جواب‌های معتبر اعلام شده در صورت پروژه مطابقت دارد – این تطابق کامل، درستی پیاده‌سازی محدودیت‌های اصلی را تأیید می‌کند. مقایسه نتایج نشان می‌دهد استفاده از Forward Checking به همراه LCV/MRV باعث می‌شود جواب بدون هیچ backtrack (یا با تعداد بسیار کم) پیدا شود، و دو پیش‌بررسی امکان‌پذیری، حالت‌های آشکارا بدون جواب را بدون صرف زمان جست‌وجو، بلافاصله رد می‌کنند.

۱۱ نکات مهم برای ارائه

در ارائه، بهتر است هر دانشجو ابتدا مسئولیت خود را توضیح دهد و سپس منطق بخش مربوط به خود را با یک مثال ساده بیان کند. تمرکز ارائه باید روی این باشد که چرا هر تکنیک برای این مسئله مناسب است و خروجی برنامه چگونه تولید می‌شود.

عضو تیم	نکات پیشنهادی برای ارائه
دانشجوا	توضیح دهد ورودی چگونه پارس و به مدل CSP تبدیل می‌شود؛ تفاوت سه محدودیت را با مثال بیان کند؛ درباره چرایی گراف همسایگی کامل و دو پیش‌بررسی امکان‌پذیری صحبت کند.
دانشجو۲	تفاوت Backtracking ساده و Forward Checking را نشان دهد؛ MRV و LCV را با مثال توضیح دهد؛ درباره نحوه شمارش Backtracks و پیچیدگی زمانی صحبت کند.

۱۲ جمع‌بندی

این پروژه نشان می‌دهد چگونه یک مسئله زمان‌بندی واقعی را می‌توان با فرمول‌بندی دقیق متغیر/دامنه/محدودیت به CSP تبدیل کرد. Backtracking Search جواب معتبر را بدون هیچ کتابخانه آماده‌ای پیدا می‌کند؛ Forward Checking با هرس زود هنگام دامنه‌ها سرعت جست‌وجو را بهتر می‌کند؛ و LCV/MRV با انتخاب هوشمندانه متغیر و مقدار، تعداد backtrackها را کاهش می‌دهند. ساختار کد به صورت ماژولار طراحی شده است؛ بنابراین هر بخش در فایل مشخصی پیاده‌سازی شده و از طریق main.py قابل اجراست. خروجی‌های به دست آمده نشان می‌دهند که پروژه مطابق صورت مسئله اجرا شده و برای ارائه کلاسی آماده است.

پیوست: خلاصهٔ توابع مهم

کاربرد	فایل	تابع / کلاس
پارس توکن‌های ورودی و نگاشت مسابقات حساس به اندیس	input_reader.py	parse_input
نرمال‌سازی جفت‌تیم حساس (مستقل از ترتیب نام)	input_reader.py	normalize_sensitive
باز کردن فایل و بازگرداندن ساختار نهایی برای main.py	input_reader.py	read_input
ساخت لیست شناسهٔ متغیرها (0 تا $N-1$)	csp_model.py	build_variables
ساخت دامنهٔ کامل (روز، ساعت، استادیوم) برای هر متغیر	csp_model.py	build_domains
ساخت گراف همسایگی کامل	csp_model.py	build_neighbors
بررسی سه محدودیت اصلی و امتیازی	csp_model.py	is_consistent
پیش‌بررسی ظرفیت کل و اصل لانه‌کبوتری	csp_model.py	is_feasible
هیوریستیک MRV	solver.py	select_unassigned_variable
هیوریستیک LCV	solver.py	order_values_lcv
هَرَس دامنهٔ همسایگان پس از هر تخصیص	solver.py	forward_check
هستهٔ بازگشتی جست‌وجو	solver.py	backtracking_search
نقطهٔ ورود حل‌کننده؛ اندازه‌گیری زمان و شمارندهٔ backtrack	solver.py	solve_csp
چاپ خروجی طبق فرمت خواسته‌شده	main.py	print_solution
اتصال تمام ماژول‌ها و اجرای کامل مسیر حل مسئله	main.py	main